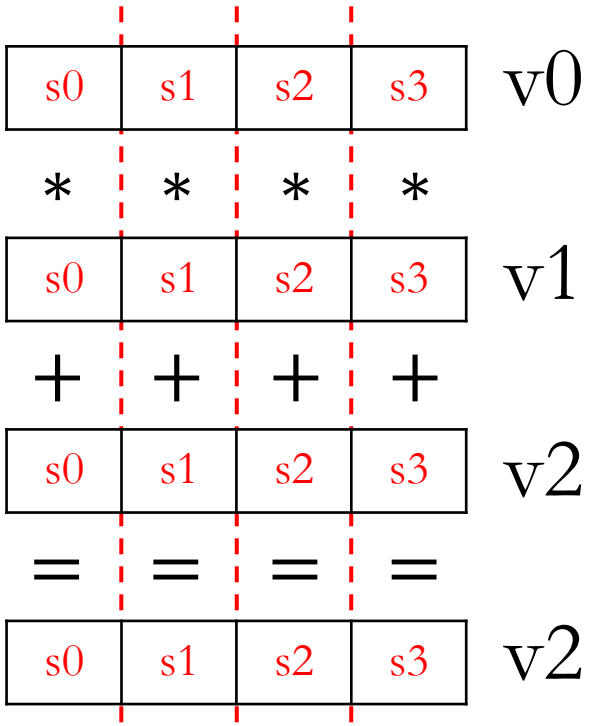
Warum ist die SIMD-Variante nicht 4-mal schneller als die skalare Variante?

```
while:
  cmp x4, xzr
  b.eq finish
  fmla v2.4s, v0.4s, v1.4s




  sub x4, x4, #4
  b while
finish:
```

```
while:
  cmp x4, xzr
  b.eq finish
  fmadd s9, s0,s1,s9
  fmadd s10,s0,s1,s10
  fmadd s11,s0,s1,s11
  fmadd s12,s0,s1,s12
  sub x4, x4, #4
  b while
finish:
```

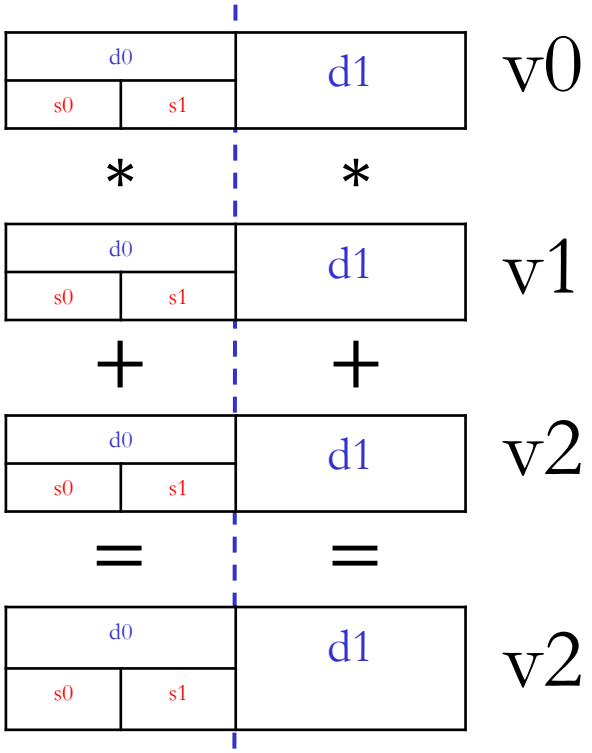
SIMD-Instruktionen

```
fmla v2.4s, v0.4s, v1.4s
```



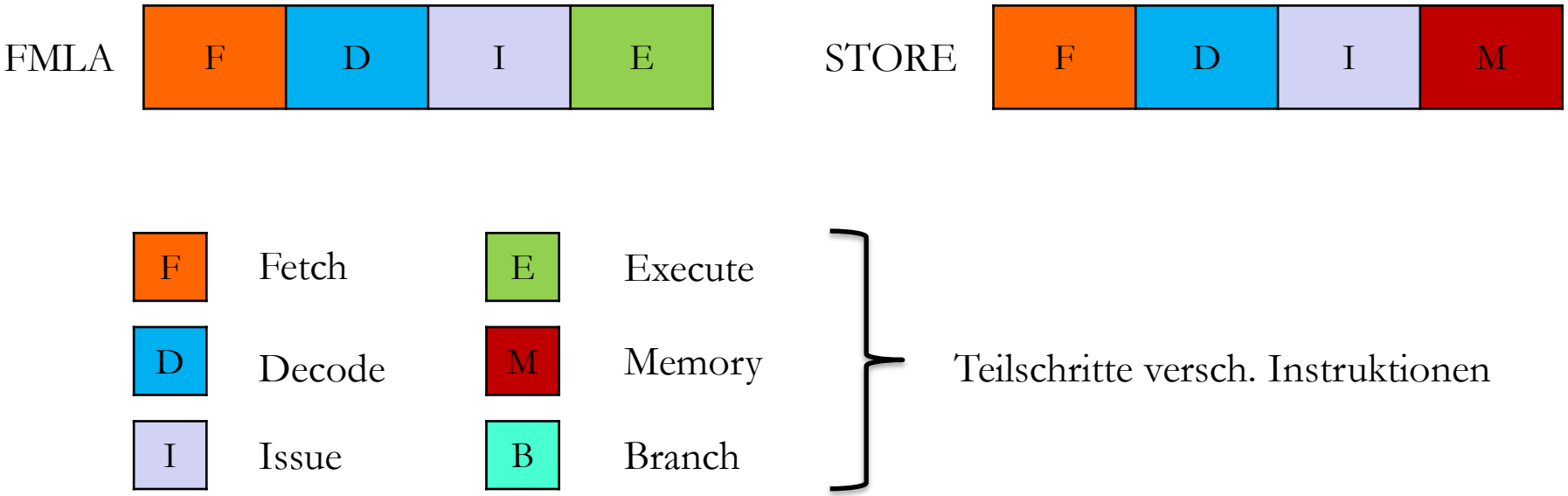
Vektorlänge 4 (float)
4 Ergebnisse/ Instruktion
8 Berechnungen/ Instruktion
(4 MUL & 4 ADD)

```
fmla v2.2d, v0.2d, v1.2d
fmla v2.2s, v0.2s, v1.2s
```



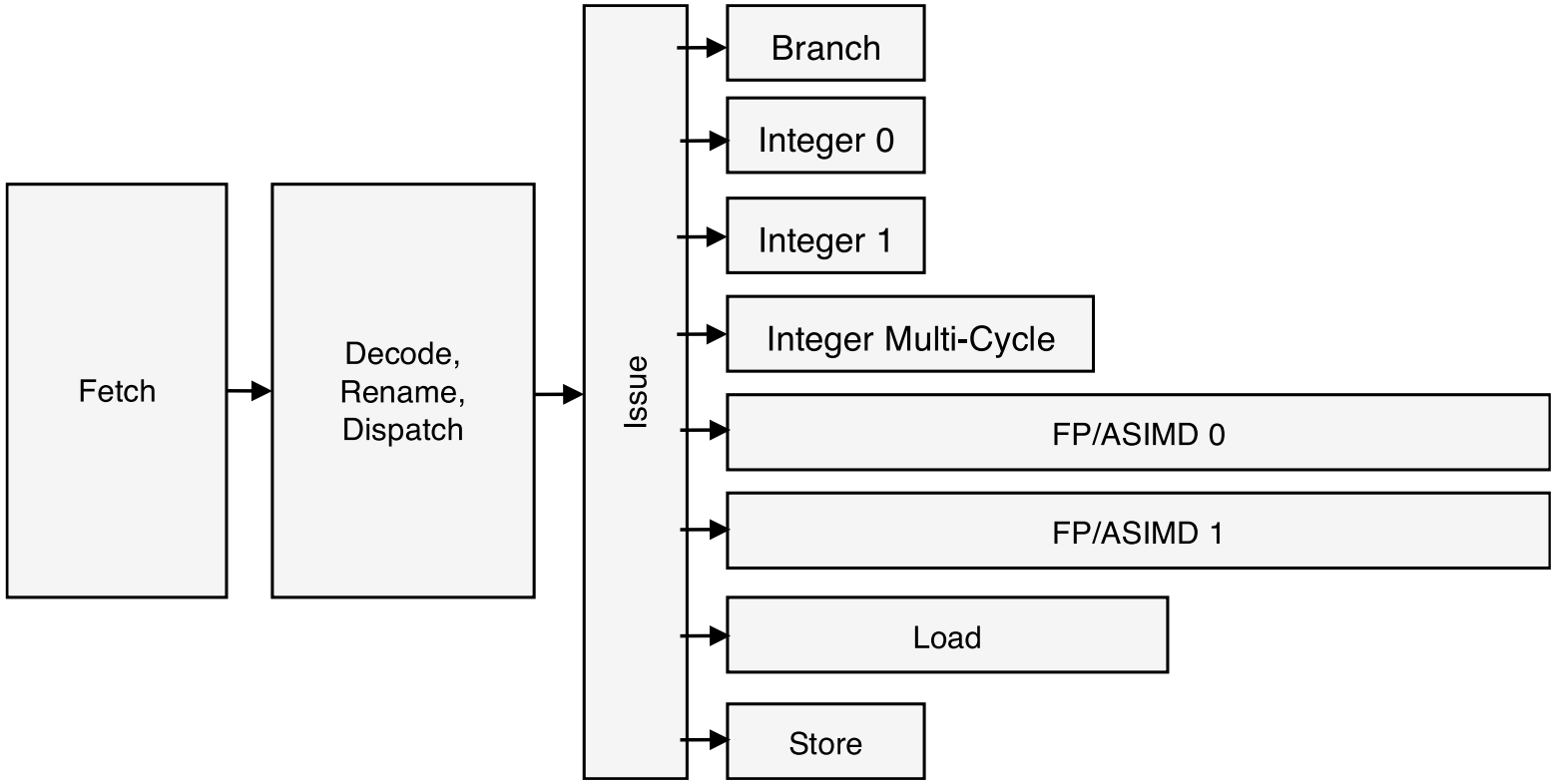
Vektorlänge 2 (float oder double)
2 Ergebnisse/ Instruktion
4 Berechnungen/ Instruktion
(2 MUL & 2 ADD)

Instruktionen ohne Pipelining



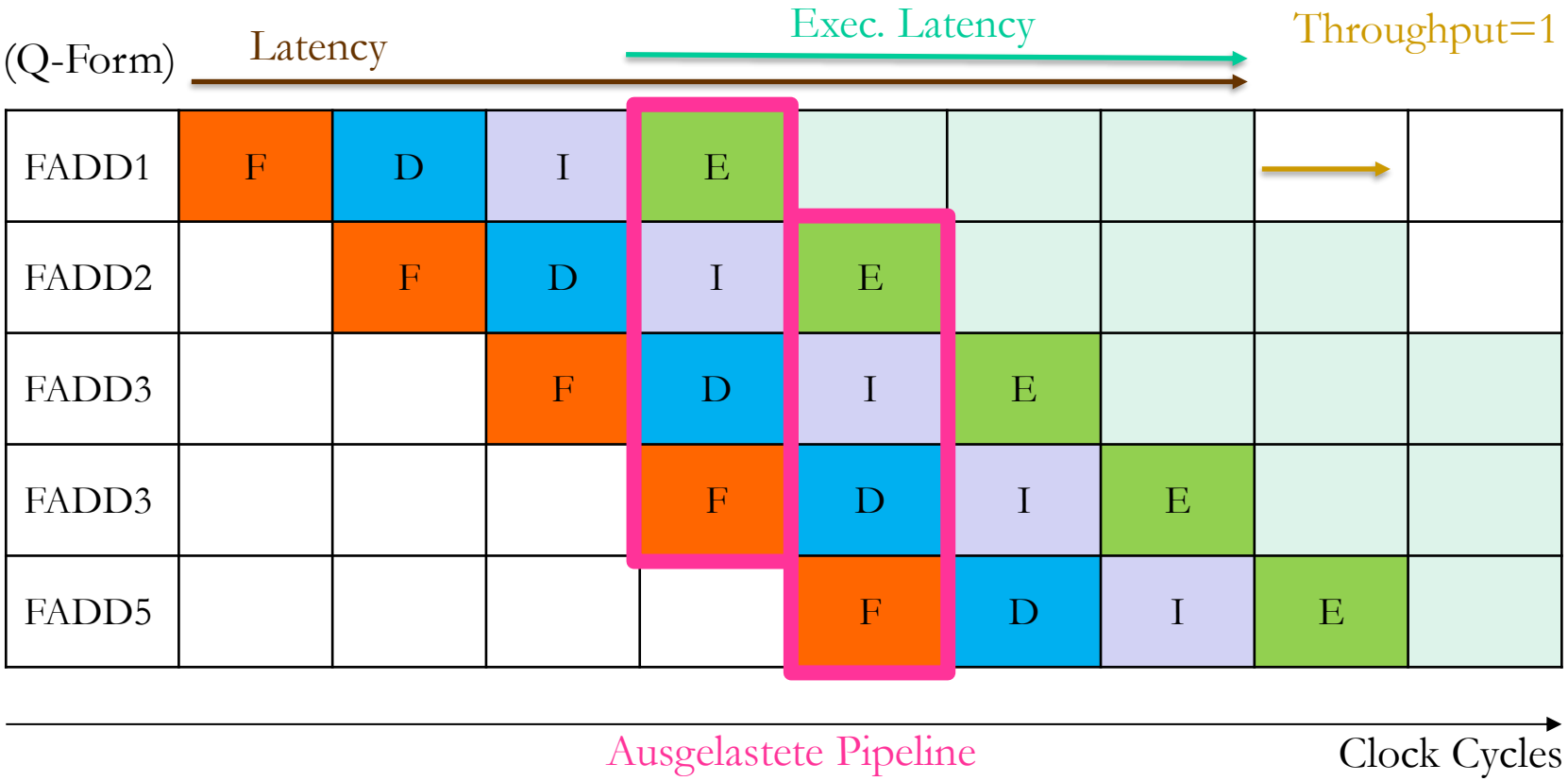
Pipeline Overview

The following diagram describes the high-level Cortex®-A72 instruction processing pipeline. Instructions are first fetched, then decoded into internal micro-operations (μops). From there, the μops proceed through register renaming and dispatch stages. Once dispatched, μops wait for their operands and issue out-of-order to one of eight execution pipelines. Each execution pipeline can accept and complete one μop per cycle.



Source: https://developer.arm.com/documentation/uan0016/a/

Instruktionen mit Pipelining

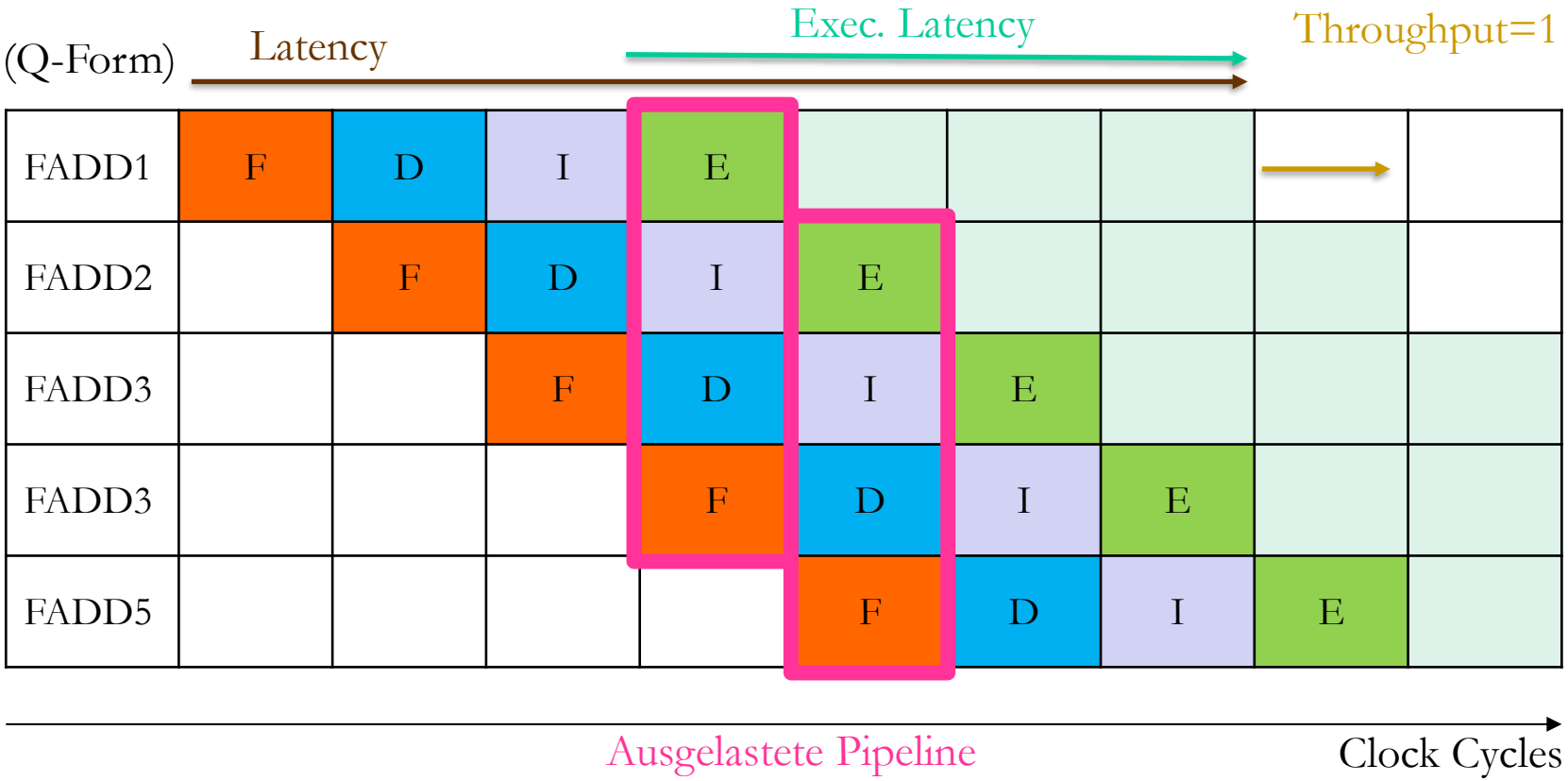


Latency – Üblicherweise die „Länge“ einer einzigen Operation; wie viele Taktzyklen dauert die Ausführung einer Instruktion insgesamt

Exec. Latency – Länge einer Operation, die nach/in der Execute Phase benötigt wird; beschreibt die Dauer bis eine nächste Operation ausgeführt werden kann, die auf Ergebnissen der aktuellen Instruktion beruht. Wichtig bei RaW Dependencies!

Throughput – Wie viele Instruktionen der gleichen Art liefern maximal pro Takt ein Ergebnis

Instruktionen mit Pipelining



Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD FP absolute value	FABS	3	2	F0/F1	
ASIMD FP arith, normal, D-form	FABD, FADD, FSUB	4	2	F0/F1	
ASIMD FP arith, normal, Q-form	FABD, FADD, FSUB	4	1	F0/F1	

Superskalare Prozessoren

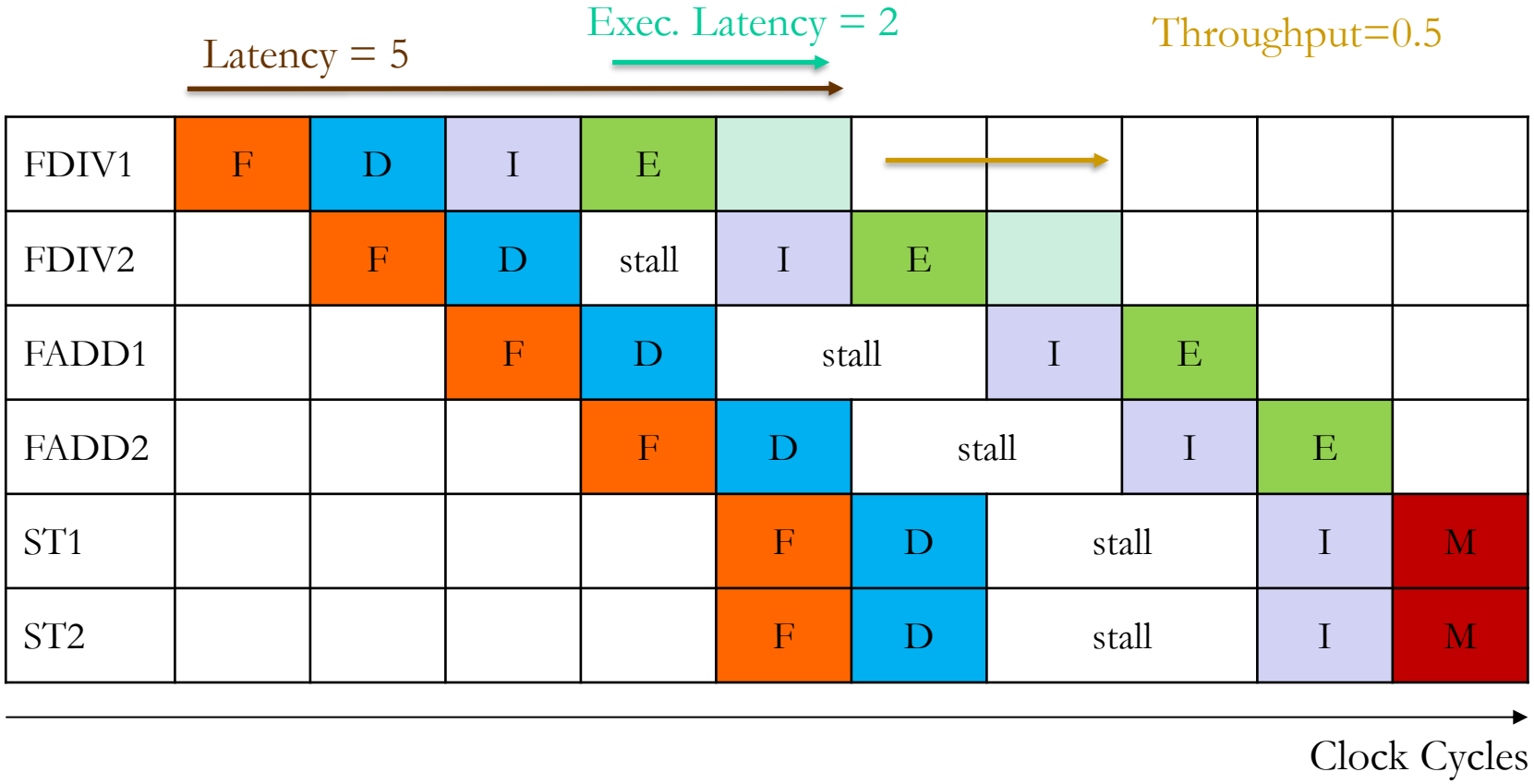
(D-Form) Latency = 7 Exec. Latency = 4 Throughput=2

FADD1	F	D	I	E				→	
FADD2	F	D	I	E				→	
FADD3		F	D	I	E				
FADD4		F	D	I	E				
FADD5			F	D	I	E			
FADD6			F	D	I	E			
FADD7				F	D	I	E		
FADD8				F	D	I	E		
FADD9					F	D	I	E	
FADD10					F	D	I	E	

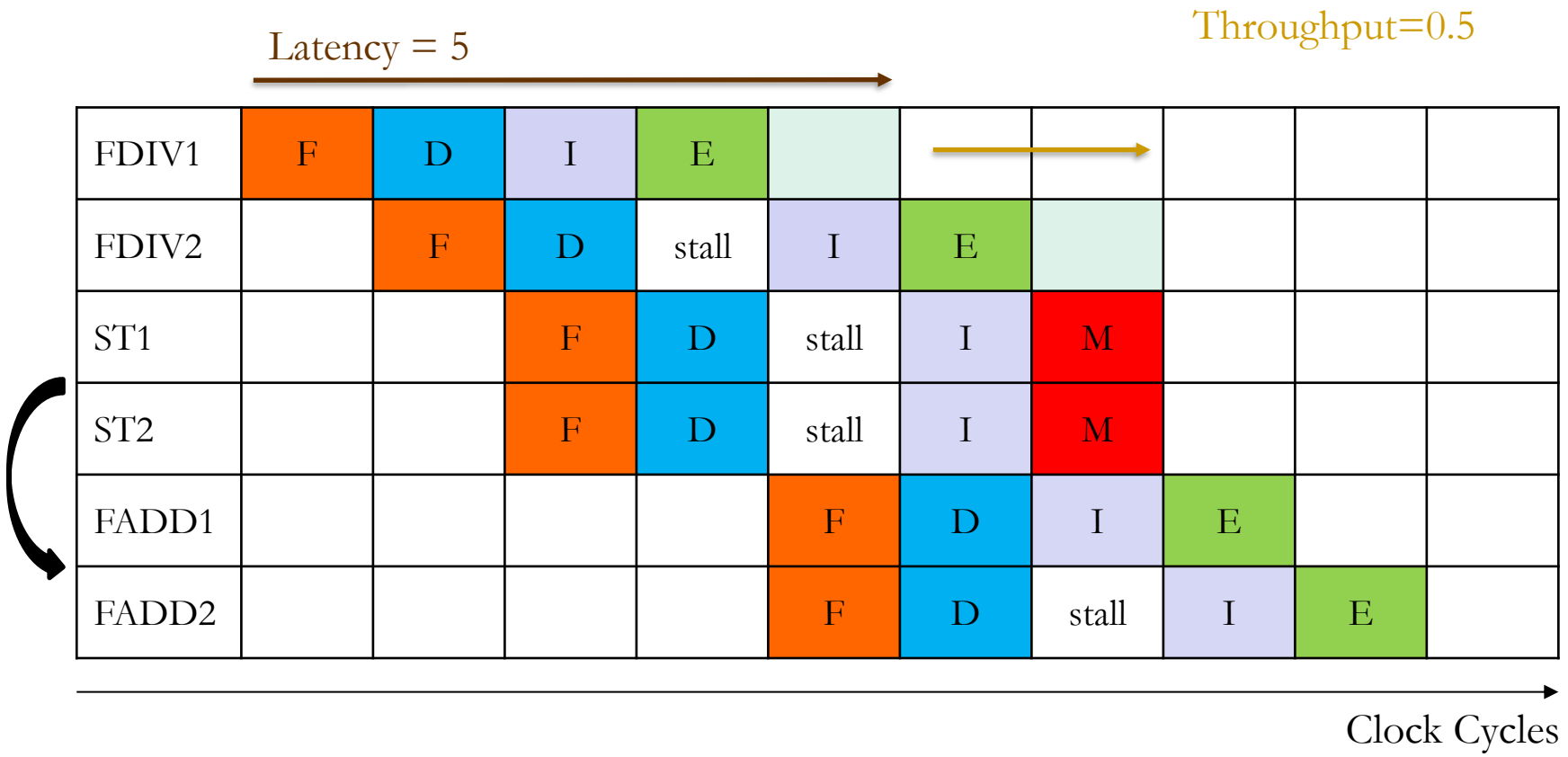
Clock Cycles

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD FP absolute value	FABS	3	2	F0/F1	
ASIMD FP arith, normal, D-form	FABD, FADD, FSUB	4	2	F0/F1	
ASIMD FP arith, normal, Q-form	FABD, FADD, FSUB	4	1	F0/F1	

In-Order-Execution und Multicycle Pipeline



Out-of-Order-Execution und Multicycle Pipeline



RaW-Dependenceis & Data Hazards

Read+Write Read Read Read after Write-Dependencies

```
fmla v0.4s, v0.4s, v1.4s
fmla v0.4s, v0.4s, v1.4s
```

RaW-Dependency
Multiplikation muss warten bis v0.4s aus vorheriger Instruktion geschrieben, dann Addition

```
fmla v2.4s, v0.4s, v1.4s
fmla v2.4s, v0.4s, v1.4s
```

RaW-Dependency
Multiplikation kann schon starten aber Addition muss warten

```
fmla v2.4s, v0.4s, v1.4s
fmla v3.4s, v0.4s, v1.4s
```

Keine RaW-Dependency

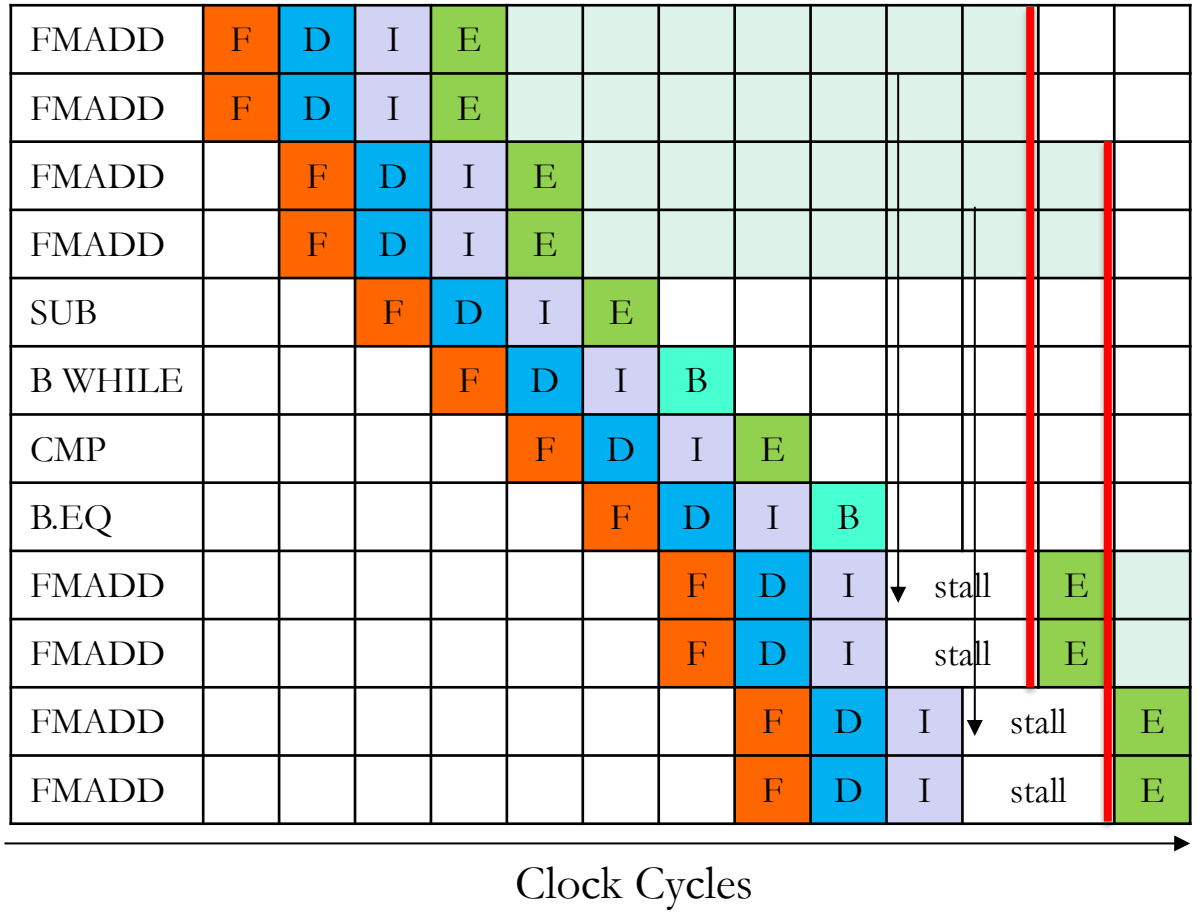
Peak-Performance

		Hardware		Instruktionen		SIMD		Pipelining
FLOPS	=	Takt	*	$\frac{elem.\,FLOP}{Instruktion}$	*	Vektorlänge	*	$\frac{Instruktion}{clock\ cycle}$
$\frac{FLOP}{s}$		Hz $=\frac{1}{s}$						Throughput $=\frac{1}{CPI}$
		1 500 000 000 Hz= $1.5*10^9$Hz =1.5 GHz		Bsp. FMLA = 2 Multiplikation und Addition gleichzeitig		Bsp. FMLA v0.4s 4 Floats gleichzeitig		Voraussetzung: Optimales Pipelining*

*keine RaW-Dependencies, Kette von Instruktionen gleichen Typs, aufgewärmte Pipeline,…

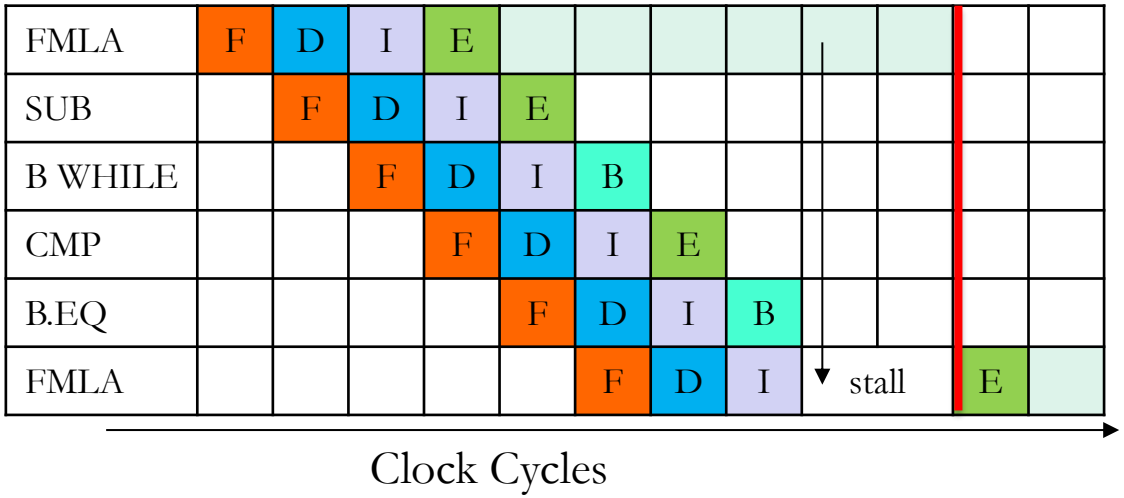
Scalar Variant vs. SIMD-Variant

```
while:
  cmp x4, xzr
  b.eq finish
  fmadd s9, s0,s1,s9
  fmadd s10,s0,s1,s10
  fmadd s11,s0,s1,s11
  fmadd s12,s0,s1,s12
  sub x4, x4, #4
  b while
finish:
```



Scalar Variant vs. SIMD-Variant

```
while:
   cmp x4, xzr
   b.eq finish
   fmla v2.4s, v0.4s, v1.4s




   sub x4, x4, #4
   b while
finish:
```



Aufgrund von RaW-Dependencies kann in beiden Fällen erst nach 10 Zyklen eine neue Schleifeniteration beginnen.